Project Title: Security Testing Secure Neighbor Discovery in Mobile Ad-hoc Network

Institution: Morgan State University, Cybersecurity Assurance and Policy Center

Submission Date: May 2026

1. Executive Summary

This project aims to validate the physical hardware implementation of the Boeing/Virginia Tech aerod/Omni code. We begin by confirming that the aerod code functions as intended in both simulated and physical environments. Next, we conduct security testing using an experimental testbed for wireless ad hoc network research. Initially, we receive the completed aerod/Omni code from Boeing and verify its performance in the Extendable Mobile Ad-hoc Network Emulator (EMANE) simulation environment. Following this, we set up an experimental testbed using hardware components. We configure the core hardware components, implement the baseline network topology, and integrate the aerod-v0.X software into the test environment.

2. Project Objective

Our primary goal is to verify the security of IPv6 Neighbor Discovery (IPv6 ND) exchanges between clients and the server in a physical hardware environment. We aim to ensure that neighbor relationships are authenticated. Specifically, we check that request signatures match by using shared secrets, timestamps for freshness, and nonces to prevent replay attacks. Our testing focuses on two main aspects: verifying signature generation with SHA-256 to ensure message integrity and rejecting tampered or expired requests. Additionally, we assess the security of the Neighbor Discovery HMAC authentication and Secure Neighbor Discovery (SEND) implementation by identifying weaknesses in signature generation and validation, evaluating replay protection, and uncovering implementation flaws. This phase aims to:

- Receive VT Hashed Message Authentication Code (HMAC) security extensions to the Aerod code base (aerod-v0.8).
- Deploy security updates on surrogate UEs and gNodeB, then bring up supporting data sets for pre-shared keys.
- Verify secured IPv6 Neighbor Discovery (IPv6 ND) exchanges between Clients and Server to ensure that neighbor relationships are secured through proper authentication.
- Perform security testing to ensure that no unauthorized entity can bypass security measures and thereby certify the authenticity and integrity of neighbor relationships.

3. Work Completed

The work completed includes verifying that all provided code works in the simulated environment, setting up the physical testbed, and performing security validation of aerod/Omni code.

3.1 Hardware & Software

A summary of the computer hardware and operating system used for security validation is listed in Table I

TABLE I: Computer Hardware and Operating System

Hardware and Software		
Node	**Hardware**	**Software**
Gateway	Geekom I9 mini-PC + BrosTrend AC1200 wireless USB adapter	Ubuntu 24.04 LTS, aerod-v0.8, tcpdump, scapy, ndisc6, Wireshark
Node1	Geekom i9 mini-PC + BrosTrend AC1200 wireless USB adapter	Ubuntu 24.04 LTS, aerod-v0.8, tcpdump, scapy, ndisc6, Wireshark
Node2	GMKtec I7 mini pc + BrosTrend AC1200 wireless USB adapter	Ubuntu 24.04 LTS, aerod-v0.8, tcpdump, scapy, ndisc6, Wireshark
Node3	GMKtec I7 mini pc + BrosTrend AC1200 wireless USB adapter	Ubuntu 24.04 LTS, aerod-v0.8, tcpdump, scapy, ndisc6, Wireshark

3.2 Hardware & Infrastructure Setup

We initially established the network in wired mode using the Ethernet port on the mini-PCs to verify the software implementation. Once the wired network was confirmed to be functioning correctly, we transitioned to a wireless setup. We configured USB 802.11 Wi-Fi dongles to operate in peer-to-peer mode and then assigned IP addresses to the wireless interfaces to activate the network. Figures 1 and 2 show the mini-PC and USB dongle used in the testbed.



Figure 1.



Figure 2.

Figure 3 presents an overview of the test environment. The testbed consists of a 5G network featuring a mini-PC running Open5GS and srsRAN, connected to a USRP B210. The mesh network includes four mini-PCs running Ubuntu 24.04, each hosting the aerod/omni server and client software. USB dongles (IEEE 802.11) operating in ad hoc or peer-to-peer mode provide physical connectivity among the devices. One device, the Gateway, runs the server software, while the other three nodes (node1, node2, and node3) run the client software.

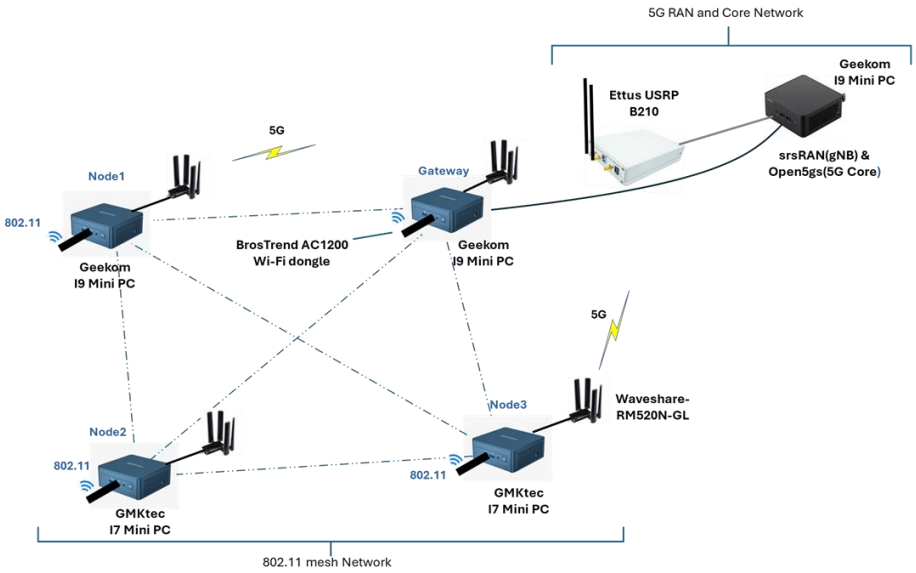


Figure 3.

The hardware, consisting of (4) I9 mini-PCs running the Ubuntu 24.04 operating system, was configured.

- Installed Aerod v-08 software and dependencies.
- Verified software functions in the EMANE simulation environment.
- Configured wireless interfaces. (See appendices A and B)
- Established physical topology for baseline testing.

3.3 Network Configuration

- Implemented code-based ad-hoc routing configuration
- Verified node discovery and peer connectivity
- Conducted initial packet transmission tests
- Verified ipv6 routing enabled

3.3 Client Software Integration

- Installed client software aerod-v0.8 on all test nodes
- Verified compatibility with system libraries
- Resolved dependency conflicts.
- Completed initial execution tests.

3.4 Initial Validation Testing

- Confirmed IP addresses were configured correctly
- Verified device connectivity via ping test
- Verified IPv6 routing enabled
- Verified that all required processes are running.

3.4.1 Mandatory Configurations

The following steps were taken before the wireless network was operational:

- Enable IPv6 forwarding across interfaces
- Enable IPv6 on the omni0 interface
- Enable Router Advertisement, although forwarding is enabled
- Ensure firewall rules don't block IPv6 traffic

Figure 4. Shows rules implemented in the /etc/sysctl.conf file.

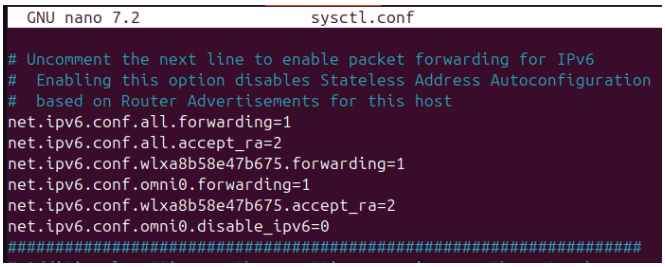


Figure 4.

3.4.2 Client Validation

- Figure 5 shows the result of the ping test verifying connectivity from node1 to node2.
- Terminal command: ***$ ping6 2001:30::5***

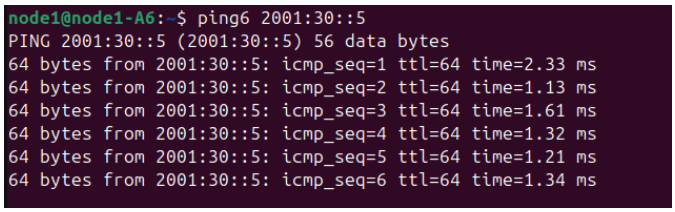


Figure 5.

- Figure 6 verifies that IPv6 is enabled on the client.
- Terminal command: ***$ ip -6 ro sh***

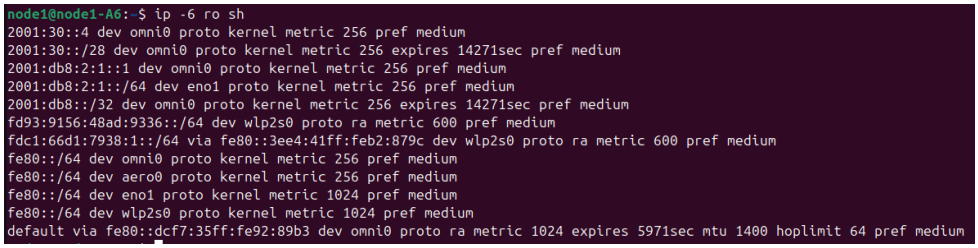


Figure 6.

- Figure 7 shows the running processes required for the Aerod software on the client.
- Terminal command : ***$ ps aux | grep aerod***

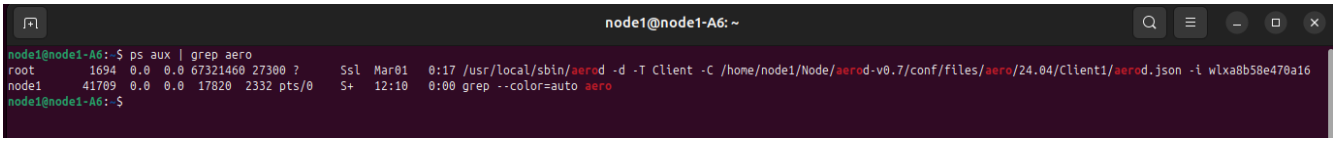


Figure 7.

4. Security Testing

The following are the tests and results we performed on the hardware implementation of the aerod-v0.8 software.

4.1 Tools & Software

The tools and software used for security testing were:

- Dell Precision 5550 Laptop running Kali Linux 2025 on VirtualBox 7.2.6
- Kali Linux Tools
  - Scapy
  - Wireshark
  - tcpdump

4.2 Security Test Cases

Before testing, we ensured that the testbed was functioning as expected. Verification showed that all nodes had routable IPv6 addresses, all IPv6 addresses responded to pings, and default routes appeared in the IPv6 routing tables of all devices. Logging of neighbor discovery drops enabled the capture of ND packets and verified ND failures in each experiment using the following commands:

$ sudo ip6tables -A INPUT -p icmpv6 –icmpv6-type neighbor-solicitation -j LOG –log-prefix "DROP NP-NS: "

$ sudo ip6tables -A INPUT -p icmpv6 –icmpv6-type neighbor-advertisement -j LOG –log-prefix "DROP NP-NA: "

The following section details the tests performed on the hardware implementation of the aerod-v0.8 software and their results. The following terminal commands were used in all tests:

- Show routing table: *$ ip -6 ro sh*
- Ping ipv6 address: *$ ping6 <ip address>*
- Flush routing table: *$ ip -6 neigh flush all*
- Show neighbor list: *$ ip -6 neigh sh*
- Send neighbor solicitation: *$ ndisc6 <ipv6 address> <interface>*
- Check iptables log: *$ journalctl -k | grep "DROP NP."*

4.2.1 Signature Generation and Validation

Signature validation testing verifies that only a client with the Private key and knowledge of the HASH algorithm can access the network. It prevents tampering and replay attacks by creating a unique, keyed signature for each request, allowing the receiver to confirm the sender's identity.

- **Key Modification**

We first verified that a client authenticates with the server. We then changed a random bit of the private key in the *aerod.c* file on the client.

```
"-----BEGIN PRIVATE KEY-----\n"
"MIIDlwIBADANBgkqhkiG9w0BAQEFAASCA4EwggN9AgEAAoHBAL4dxXzE10fWPeTx\n"
"no5W620QFWGTJi0yxr73PO9u+1zPAi+XO/uhJtM5hAIsKHfxvE29imeENy270HdF\n"
```

Figure 8.

We then recompiled the *aerod* code and attempted to join the network. The neighbor cache of a target device (node1) is flushed. Forcing a neighbor advertisement (NA) from node2 involves sending a neighbor solicitation (NS) from node1 to node2 to request node1's MAC address.

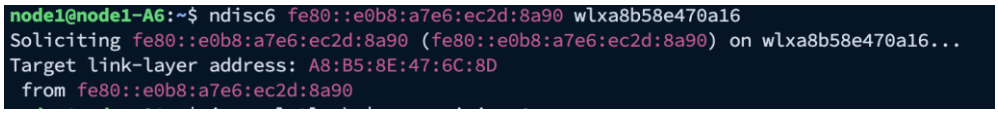


Figure 9.

The experiment then observes the results of the neighbor cache update and the ping test.

Result:

Node2 cannot access the network without the correct private key (NA packets are discarded at node1).

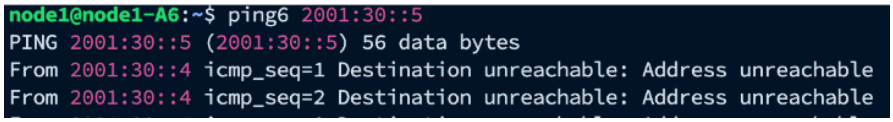


Figure 10.

4.2.2 Replay Attack Testing

Replay attack testing validates system security against attackers who capture and resend valid data packets, such as authentication requests, to gain unauthorized access. Testing involves intercepting legitimate traffic and retransmitting it to verify whether the server improperly accepts duplicate commands.

1. ***Replay Same Request***

This test sends an identical signed request twice. The test repeats the previous steps by flushing the target device's (node1) neighbor cache and forcing an NA from node2. Scapy Python scripts capture the NA from node2. After flushing node1's cache again, the test replays the captured NA message. The process observes the neighbor cache updates and conducts ping tests to verify the results

Result:

- The second request is rejected.
- Neighbor cache does not update

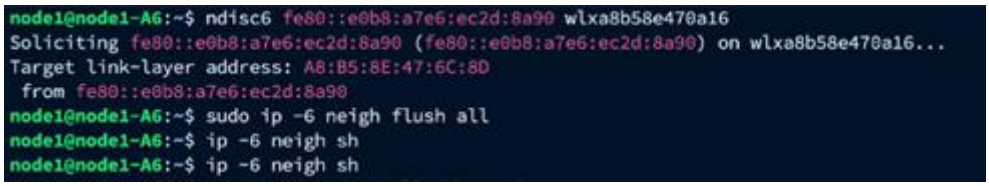


Figure 11.

2. *Replay with Old Timestamp*

The test repeats the previous steps by flushing the target device's (node1) neighbor cache and forcing an NA from node2. Scapy Python scripts capture node2's NA response. The procedure sets the timestamp to an earlier value (-10 minutes), flushes node1's cache again, and replays the captured NA message. The process then observes the neighbor cache update and ping test results.

Result:

- Message rejected outside the allowed window.
- Node2 cannot access the network without a valid timestamp
- NA packets dropped at node 1.

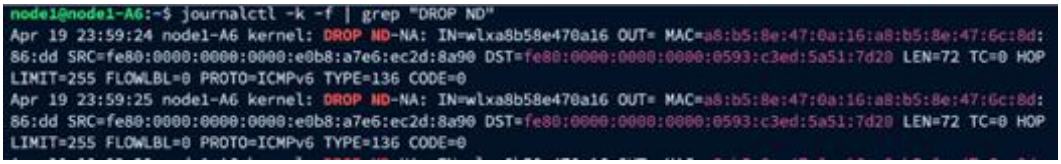


Figure 12.

3. *Replay Nonce Reuse*

The test repeats the process by flushing node1's neighbor cache and forcing an NA from node2. Scapy Python scripts capture the NS request from node1, extract the nonce, and craft an NA response packet using the nonce. The procedure flushes node1's cache, sends the crafted NA message, observes the neighbor cache update, and then conducts a ping test.

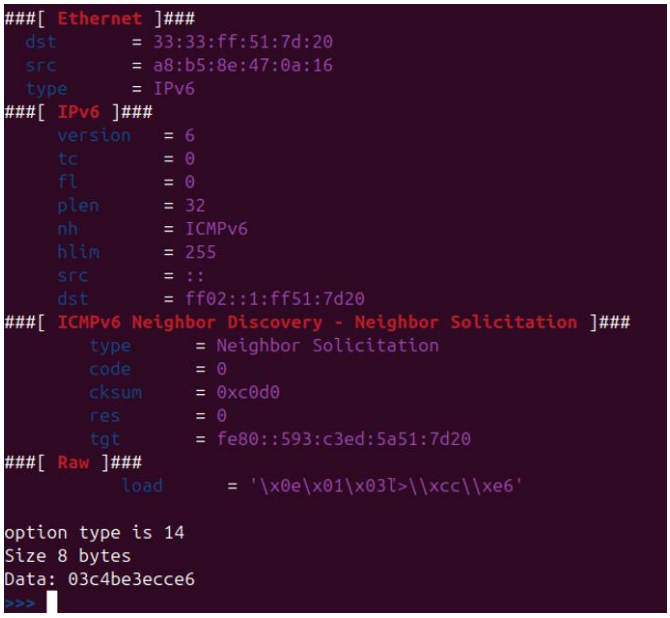


Figure 13.

Result:

- The message is rejected
- Node2 cannot access the network without a valid nonce.
- NA packets are dropped at node 1.

5. Challenges & Mitigation

Issue	Impact	Mitigation
Outdated driver for USB adapter	Delayed configuration	Updated firmware
Race conditions between the omni0 interface being up and the start of the DHCP6 server on hardware not visible in the simulation	Kea-dhcp6 server failed to start	We added a check to ensure the omni0 interface is up before starting the kea-dhcp6 server in the rc. local start-up script.
Visibility into logs is needed for troubleshooting	Delayed configuration	Enabled logging for Radvd and kea-dhcp6 server.
Confirm packet discard	Difficult to confirm the expected result	Created a firewall rule and enabled logging.

6. Conclusion

This project evaluated the security and resilience of the secure neighbor discovery mechanism within the aerod-v.08 code running on Ubuntu 24.04 hardware. The primary objective was to determine whether an unauthorized entity can circumvent the security mechanisms of the neighbor discovery protocol and gain access to the mobile Ad hoc network. Throughout the testing process, several test cases were run, including tests of various forms of replay-attack resistance.

In this project, we validated the operation of the aerod/Omni code on physical hardware. We successfully demonstrated the resistance of both the HMAC and SEND variants of the *aerod-v0.8* code to replay attack. We confirmed that devices needed to possess the correct keys to communicate with the network's nodes. We also confirmed the proper use of nonces, timestamps, and signatures in the *aerod* code to ensure message freshness and authenticity.

Overall, the project demonstrates the efficacy of security controls in device-to-device communication in a mobile ad hoc network. It provides an effective baseline for neighbor discovery and underscores the need for robust authentication to protect decentralized wireless networks from unauthorized access.

APPENDIX A

Server Start-Up Script

```
#!/bin/sh
# --- Core system setup ---
ulimit -S -c unlimited > /dev/null 2>&1
/usr/sbin/rsyslogd

# --- Start aerod ---
/usr/local/sbin/aerod -d -T Proxy \
  -C /home/cream/Node/aerod/conf/files/aero/24.04/Proxy/aerod.json &
# --- Wait for omni0 to have an IPv6 link-local address ---
echo "Waiting for omni0 to come up with link-local IPv6..."
while ! ip -6 addr show dev omni0 | grep -q 'fe80'; do
    sleep 2
done
echo "omni0 is up with link-local IPv6."

# --- Start radvd ---
pkill radvd 2>/dev/null
rm -f /usr/local/var/run/radvd.pid
# Start radvd with logging to stderr (captured to file)
/usr/local/sbin/radvd -C /home/cream/Node/aerod/conf/files/aero/24.04/Proxy/radvd.conf -m stderr >> /var/log/radvd.log 2>&1 &

# Verify radvd startup
sleep 1
if pgrep -x radvd >/dev/null; then
    echo "$(date): radvd started successfully" >> /var/log/radvd.log
else
    echo "$(date): radvd failed to start!" >> /var/log/radvd.log
fi

# --- Prepare KEA DHCP6 directories ---
rm -rf /usr/local/var/run/kea /var/run/kea
mkdir -p /var/run/kea
ln -s /var/run/kea /usr/local/var/run/kea

# --- Start KEA DHCP6 ---
/usr/local/sbin/kea-dhcp6 \
  -c /home/cream/Node/aerod/conf/files/aero/24.04/Proxy/kea-dhcp6.conf \
  >> /var/log/kea-dhcp6.log 2>&1 &

# --- Start SSH ---
mkdir -p /run/sshd
chmod 0755 /run/sshd
/usr/sbin/sshd -o PermitRootLogin=yes

exit 0
```

APPENDIX B

Client Start-Up Script

```
!/bin/sh

#wireless startup script
sudo nmcli connection add type wifi con-name Boeing_Adhocnet
ifname wlxa8b58e470a16 autoconnect yes ssid Boeing_Adhocnet mode
adhoc
sudo nmcli connection modify Boeing_Adhocnet ipv4.method manual
ipv4.addresses 10.0.0.2/24
sudo nmcli con up Boeing_Adhocnet

#Client startup script
/usr/local/sbin/aerod -d -T Client -C /home/node1/Node/aerod-
v0.7/conf/files/aero/24.04/Client1/aerod.json -i wlxa8b58e470a16

exit 0
```

APPENDIX C

Replay Same Request Script

```
from scapy.all import *

INTERFACE = "wlxa8b58e476c8d"
print("waiting for an NA to capture...")

####Capture and saves NA packett###########
pkt = sniff(iface=INTERFACE, filter="icmp6 && ip6[40] == 136", count=1)[0]
print(f"Captured NA from {pkt[IPv6].src}")
pkt.show()
wrpcap("ndisctest.pcap", pkt)

#### Reads and replays packet ###########
packet = rdpcap("/home/node2/ndisctest.pcap")
pkt = packet[0]
print("Replaying packet....")
pkt.show()
sendp(pkt, inter=1, loop=0, count=1, iface=INTERFACE)
```

APPENDIX D

Replay Old Timestamp Script

```
from scapy.all import *
from time import time

def replay_modified_na(interface="wlxa8b58e476c8d"):
    # Capture NA packet
    na_filter = "icmp6 and ip6[40] == 136"
    pkt = sniff(iface=interface, filter=na_filter, count=1)[0]

    # Modify timestamp in packet to 1hr ago
    pkt.time = int(time()) - 3600

    # Clear cached checksums/lengths. Scapy recalculates these
    if pkt.haslayer(IPv6):
        del pkt[IPv6].payload_len
    if pkt.haslayer(ICMPv6ND_NA):
        del pkt[ICMPv6ND_NA].chksum

    # Replay packet
    sendp(pkt, iface=interface, verbose=True)

if __name__ == "__main__":
    replay_modified_na()
```

APPENDIX E

Replay Nonce Script

```
from scapy.all import *
from scapy.layers.inet6 import IPv6, ICMPv6ND_NS, ICMPv6NDOptUnknown

def extract_nonce(packet):
    # Check for IPv6 and ICMPv6 Neighbor Solicitation
    if packet.haslayer(IPv6) and packet.haslayer(ICMPv6ND_NS):
        print(f"Captured NS from: {packet[IPv6].src}")

        # Loop through options to find the Nonce option (Type 14)
        current_layer = packet[ICMPv6ND_NS].payload
        while current_layer:
            if isinstance(current_layer, ICMPv6NDOptUnknown) and current_layer.type == 14:
                nonce_value = current_layer.data
                print(f"Extracted Nonce: {nonce_value.hex()}")
                return packet, nonce_value
            current_layer = current_layer.payload
    return None, None

# Sniff network traffic until a valid NS packet with a nonce is found
print("Sniffing for IPv6 NS packets...")
target_packet, extracted_nonce = sniff(iface="wlxa8b58e476c8d", filter="icmp6", lfilter=lambda p: extract_nonce(p)[1] is not None, count=1)[0],
None, extracted_nonce = extract_nonce(target_packet)

# Define network parameters based on the captured NS
local_mac = "a8:b5:8e:47:6c:8d"
target_ip = target_packet[IPv6].src
target_mac = target_packet[Ether].src
solicited_ip = target_packet[ICMPv6ND_NS].tgt

# Build Ethernet and IPv6 layers
eth_layer = Ether(src=local_mac, dst=target_mac)
ip_layer = IPv6(src=solicited_ip, dst=target_ip)

# Build ICMPv6 Neighbor Advertisement layer
na_layer = ICMPv6ND_NA(tgt=solicited_ip, R=0, S=1, O=1)

# Build Target Link-Layer Address option (Type 2)
tgt_ll_address = ICMPv6NDOptDstLLAddr(lladdr=local_mac)

# Build the Nonce option (Type 14) using the extracted data
nonce_option = ICMPv6NDOptUnknown(type=14, len=len(extracted_nonce)//8 + 1, data=extracted_nonce)

# Stack layers and send the crafted packet
replay_packet = eth_layer / ip_layer / na_layer / tgt_ll_address / nonce_option
sendp(replay_packet, iface="wlxa8b58e476c8d")
print("Replayed NA packet with matching nonce successfully.")
```